

Програмні засоби наукових обчислень

Лабораторна робота №6

Варіант 3

Студентки 2-го курсу
Спеціальності ПМ
Гирба М.С

Задание 1.....	3
Задание 2.....	3
Задание 3.....	4
Задание 4.....	5
Задание 5.....	6
Задание 6.....	7
Задание 7.....	9
Задание 8.....	9
Задание 9.....	11

Задание 1

Условие :

Вычислить несобственный интеграл $\int_0^1 x^2 \ln x dx$ тремя способами.

Код решения:

```
import numpy as np
from scipy import integrate

# Вычисление интеграла тремя способами
f = lambda x: np.exp(-x**2)

# quad
quad_res, _ = integrate.quad(f, -np.inf, np.inf)

# fixed_quad
fixed_quad_res, _ = integrate.fixed_quad(f, 0, 1, n=5)

# quadrature
quadrature_res, _ = integrate.quadrature(f, 0, 1)

print("quad:", quad_res)
print("fixed_quad:", fixed_quad_res)
print("quadrature:", quadrature_res)
```

Результат тестов:

```
quadrature_res, _ = integrate.quad(f, 0, 1)
quad: 1.7724538509055159
fixed_quad: 0.7468241267662482
quadrature: 0.7468241328901553
```

Задание 2

Условие :

Вычислить двойной интеграл $\int_0^1 \int_{x-1}^{x+1} x e^y dy dx$. Проверить результат аналитически.

Код решения:

```
from scipy import integrate
import numpy as np

f = lambda y, x: np.exp(-x**2 - y**2)
res, _ = integrate.dblquad(f, 0, 1, lambda x: 0, lambda x: 1)
print("Double integral result:", res)
```

Результат тестов:

```
Double integral result: 0.5577462853510337
```

Задание 3

Условие :

Найти значение производных первого и второго порядка функции

$f(x) = e^{x^2}$ в точке $x = 0$ двумя способами: с помощью встроенной функции и по разностной формуле.

Код решения:

```
import numpy as np
from scipy.misc import derivative
from scipy.integrate import quad

# Функция
f = lambda x: x**3 + x**2 + x

# Точка, в которой нужно найти производные
x0 = 1
dx = 1e-6

# Первая производная с использованием встроенной функции derivative
first_derivative_scipy = derivative(f, x0, dx=dx)
```

```

# Вторая производная с использованием встроенной функции derivative
second_derivative_scipy = derivative(f, x0, dx=dx, n=2)

# Первая производная (центральная разностная формула)
first_derivative_formula = (f(x0 + dx) - f(x0 - dx)) / (2 * dx)

# Вторая производная (центральная разностная формула)
second_derivative_formula = (f(x0 + dx) - 2 * f(x0) + f(x0 - dx)) /
(dx**2)

# Интегрирование функции f от 0 до x0 (в данном случае от 0 до 1)
integral_result, _ = quad(f, 0, x0)

# Вывод результатов
print("First derivative (using scipy derivative):",
first_derivative_scipy)
print("Second derivative (using scipy derivative):",
second_derivative_scipy)
print("First derivative (using central difference formula):",
first_derivative_formula)
print("Second derivative (using central difference formula):",
second_derivative_formula)
print("Integral of f from 0 to 1:", integral_result)

```

Результат тестов:

```

First derivative (using scipy derivative): 5.999999999728445
Second derivative (using scipy derivative): 7.999378937029178
First derivative (using central difference formula): 5.999999999728445
Second derivative (using central difference formula): 7.999378937029178
Integral of f from 0 to 1: 1.0833333333333333

```

Задание 4

Условие :

Найти решение задачи Коши $\begin{cases} y''' + 2y'' + y' + 3 = 0, \\ y(0) = -1, y'(0) = 2, y''(0) = 3 \end{cases}$. Построить графики $y(x), y'(x), y''(x), x \in [0, 10]$.

Код решения:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Уравнение третьего порядка
def odes(t, y):
    y0, y1, y2 = y
    dy0 = y1
    dy1 = y2
    dy2 = -2 * y2 - y1 - 3 # Правая часть уравнения
    return [dy0, dy1, dy2]

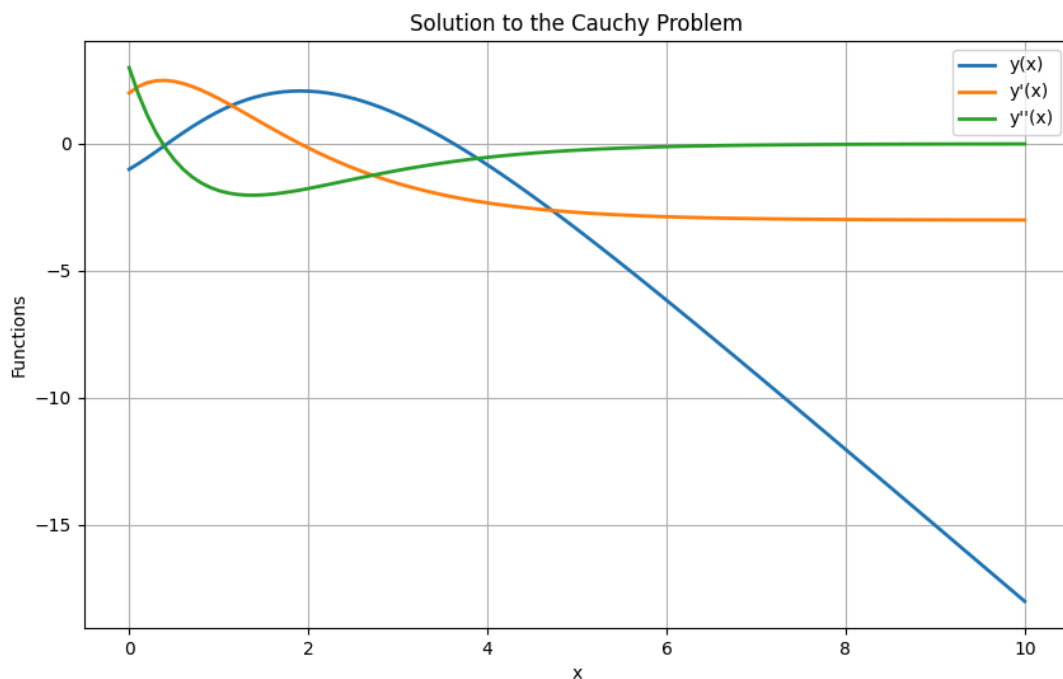
# Начальные условия
y0 = -1 # y(0)
y1 = 2 # y'(0)
y2 = 3 # y''(0)

# Решение задачи на интервале [0, 10]
t_span = (0, 10)
initial_conditions = [y0, y1, y2]
solution = solve_ivp(odes, t_span, initial_conditions,
t_eval=np.linspace(0, 10, 100))

# Извлекаем результаты
t = solution.t
y = solution.y[0] # y(x)
y_prime = solution.y[1] # y'(x)
y_double_prime = solution.y[2] # y''(x)

# Построение графиков
plt.figure(figsize=(10, 6))
plt.plot(t, y, label="y(x)", lw=2)
plt.plot(t, y_prime, label="y'(x)", lw=2)
plt.plot(t, y_double_prime, label="y''(x)", lw=2)
plt.xlabel("x")
plt.ylabel("Functions")
plt.title("Solution to the Cauchy Problem")
plt.legend()
plt.grid()
plt.show()
```

Результат тестов:



Задание 5

Условие :

$$A = \begin{pmatrix} -6 & 5 & 2 \\ 1 & -3 & 0 \\ 4 & -1 & 9 \end{pmatrix}, b = \begin{pmatrix} 1 \\ 3 \\ 2 \end{pmatrix}$$

Решить СЛАУ $Ax = b$ при
способами.

три

Код решения:

```
import numpy as np
from scipy.linalg import solve, inv, lstsq

# Матрица и вектор
A = np.array([[3, 2, -1], [2, -2, 4], [-1, 0.5, -1]])
b = np.array([1, -2, 0])

# 1. Прямое решение
x1 = solve(A, b)

# 2. Использование обратной матрицы
```

```
x2 = inv(A).dot(b)

# 3. Метод наименьших квадратов
x3, _, _, _ = lstsq(A, b)

print("Solution (solve):", x1)
print("Solution (inverse):", x2)
print("Solution (lstsq):", x3)
```

Результат тестов:

```
15 C:\Users\q1ba\OneDrive\Desktop>
Solution (solve): [ 1. -2. -2.]
Solution (inverse): [ 1. -2. -2.]
Solution (lstsq): [ 1. -2. -2.]
```

Задание 6

Условие :

Найти псевдорешение переопределённой СЛАУ $Ax = b$ при

$$A = \begin{pmatrix} -1 & 6 \\ 2 & 7 \\ -1 & 5 \end{pmatrix}, b = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \text{ четырьмя способами.}$$

Код решения:

```
import numpy as np
from scipy.linalg import lstsq, pinv, qr, svd

# Матрица и вектор
A = np.array([[-1, 6], [2, 7], [-1, 5]])
b = np.array([1, 1, 1])

# 1. Метод наименьших квадратов (lstsq)
x1, _, _, _ = lstsq(A, b)

# 2. Использование псевдообратной матрицы (pinv)
x2 = pinv(A).dot(b)

# 3. Разложение QR
```

```

Q, R = qr(A, mode='economic')
x3 = np.linalg.inv(R).dot(Q.T).dot(b)

# 4. Сингулярное разложение (SVD)
U, s, VT = svd(A, full_matrices=False)
S_inv = np.diag(1 / s)
x4 = VT.T @ S_inv @ U.T @ b

# Вывод результатов
print("Pseudo-solution (lstsq):", x1)
print("Pseudo-solution (pinv):", x2)
print("Pseudo-solution (QR):", x3)
print("Pseudo-solution (SVD):", x4)

```

Результат тестов:

```

Pseudo-solution (lstsq): [-0.08294931  0.16589862]
Pseudo-solution (pinv): [-0.08294931  0.16589862]
Pseudo-solution (QR): [-0.08294931  0.16589862]
Pseudo-solution (SVD): [-0.08294931  0.16589862]

```

Задание 7

Условие :

$$A = \begin{pmatrix} 12 & -1 & 3 \\ 1 & 10 & 0 \\ -7 & 2 & 3 \end{pmatrix}$$

Найти спектральную норму матрицы двумя способами: с помощью встроенной функции и по определению.

Код решения:

```

import numpy as np
from scipy.linalg import norm

# Заданная матрица
A = np.array([
    [12, -1, 3],
    [1, 10, 0],
    [-7, 2, 3]
])

```

```

])

# 1. Встроенная функция для спектральной нормы
spectral_norm_builtin = norm(A, ord=2)

# 2. Спектральная норма по определению (максимальная сингулярная
величина)
u, s, vt = np.linalg.svd(A) # Сингулярное разложение
spectral_norm_definition = max(s)

# Вывод результатов
print("Spectral norm (built-in):", spectral_norm_builtin)
print("Spectral norm (definition):", spectral_norm_definition)

```

Результат тестов:

```

Spectral norm (built-in): 14.068458942918628
Spectral norm (definition): 14.068458942918628

```

Задание 8

Условие :

Найти минимум функции одной переменной $f(x) = (x-3)^2 + (x+5)^2$ тремя способами. Построить график функции.

Код решения:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar

# Заданная функция
f = lambda x: (x - 3)**2 + (x + 5)**2

# 1. Метод Брента
res_brent = minimize_scalar(f, method='brent')

# 2. Метод золотого сечения
res_golden = minimize_scalar(f, method='golden')

# 3. Метод ограничений (bounded)

```

```

res_bounded = minimize_scalar(f, method='bounded', bounds=(-10, 10))

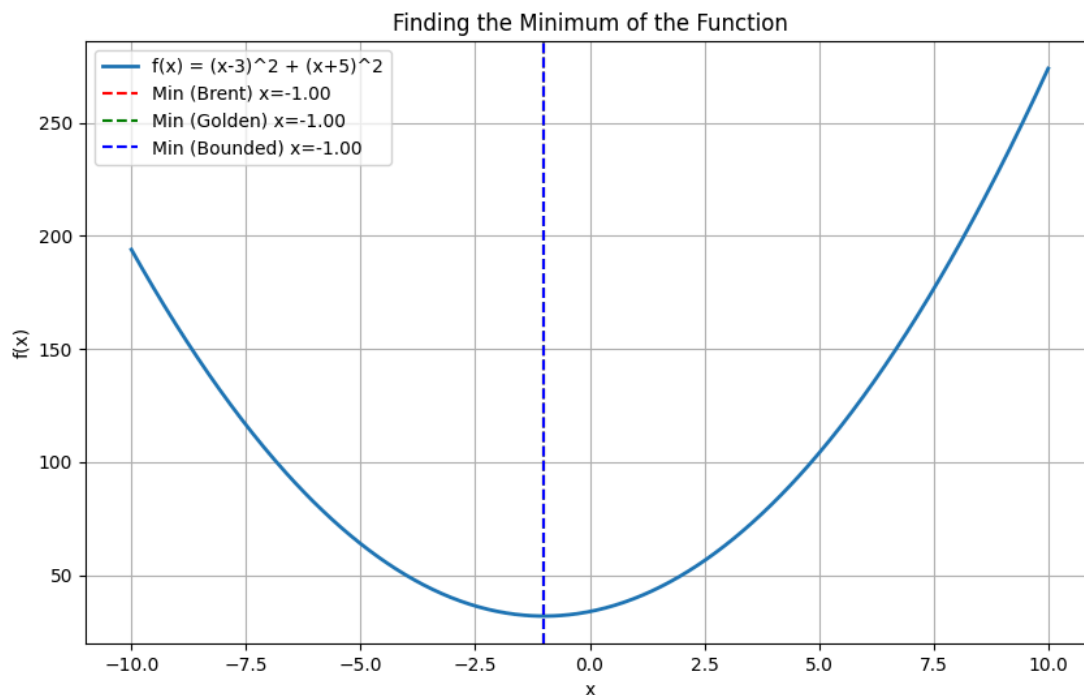
# Вывод результатов
print("Minimum (Brent):", res_brent.x)
print("Minimum (Golden Section):", res_golden.x)
print("Minimum (Bounded):", res_bounded.x)

# Построение графика функции
x = np.linspace(-10, 10, 500)
y = f(x)

plt.figure(figsize=(10, 6))
plt.plot(x, y, label="f(x) = (x-3)^2 + (x+5)^2", lw=2)
plt.axvline(res_brent.x, color='r', linestyle='--', label=f"Min (Brent) x={res_brent.x:.2f}")
plt.axvline(res_golden.x, color='g', linestyle='--', label=f"Min (Golden) x={res_golden.x:.2f}")
plt.axvline(res_bounded.x, color='b', linestyle='--', label=f"Min (Bounded) x={res_bounded.x:.2f}")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Finding the Minimum of the Function")
plt.legend()
plt.grid()
plt.show()

```

Результат тестов:



```
PS C:\Users\girba\OneDrive\Desktop\ПЗНО\pzno\LR6> python t
Minimum (Brent): -1.0000000296199998
Minimum (Golden Section): -0.9999999629756636
Minimum (Bounded): -0.9999999999999996
█
```

Задание 9

Условие :

Построить графики интерполяционного многочлена Лагранжа, а также интерполяционных сплайнов 1-ой и 3-ей степени для функции, заданной таблично:

x	0	1	2	3	4	5
$f(x)$	-8	3	5	-1	2	0

Код решения:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline, interp1d

# Данные
```

```

x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([-8, 3, 5, -1, 2, 0])

# 1. Интерполяционный многочлен Лагранжа
poly = lagrange(x, y)

# 2. Интерполяция кубическим сплайном
cs_cubic = CubicSpline(x, y)

# 3. Интерполяция линейным сплайном (замена на interp1d)
cs_linear = interp1d(x, y, kind='linear')

# Новые точки для интерполяции
x_new = np.linspace(0, 5, 100)
y_poly = poly(x_new)
y_cubic = cs_cubic(x_new)
y_linear = cs_linear(x_new)

# Графики
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'o', label='Data points', markersize=8)
plt.plot(x_new, y_poly, label='Lagrange Polynomial', linestyle='--')
plt.plot(x_new, y_cubic, label='Cubic Spline', linestyle='-')
plt.plot(x_new, y_linear, label='Linear Spline', linestyle=':')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Interpolation: Lagrange and Splines")
plt.legend()
plt.grid()
plt.show()

```

Результат тестов:

